

UNIVERSIDAD DE LAS FUERZAS ARMADAS

“ESPE”



DEPARTAMENTO DE CIENCIAS DE LA COMPUTACIÓN

INGENIERÍA EN ANÁLISIS Y DISEÑO DE SOFTWARE

CARRERA DE INGENIERÍA EN SOFTWARE



NOMBRE:

- Blacio Machuca Julio César
- Sigcha Palma Kenned Jhair
- Solano Sanchez Josue Vital
- Toapanta Jama Alexander Jhosue

NRC: 30724

FECHA: 28/04/2026

DOCENTE: Ing. Jenny Ruiz Robalino

GUÍA 01

- Objetivo

Analizar y modelar los requerimientos funcionales de un sistema CRUD de estudiantes mediante la elaboración de diagramas de casos de uso en PlantUML, aplicando principios de análisis y diseño de software orientado a objetos, con el fin de representar de manera estructurada la interacción entre actores y funcionalidades del sistema, garantizar la trazabilidad de requisitos y establecer una base sólida para las siguientes etapas de desarrollo, como el diseño estructural y dinámico del sistema.

- Desarrollo
 - Requisitos Funcionales

RF	Caso de uso	Actor	Prioridad	Criterio de aceptación	Evidencia PlantUML
RF-01	Agregar estudiante	Administrador académico	Alta	El sistema guarda un estudiante con ID único, Nombre obligatorio y Edad válida.	UC1
RF-02	Actualizar estudiante	Administrador académico	Alta	El sistema modifica Nombre y/o Edad luego de validar campos obligatorios.	UC2
RF-03	Eliminar estudiante	Administrador académico	Media	El sistema elimina o marca como eliminado al estudiante seleccionado.	UC3
RF-04	Mostrar todo	Administrador académico	Alta	El sistema muestra una tabla con las columnas ID, Nombre y Edad.	UC4

RF-05	Validar datos del estudiante	Sistema	Alta	El sistema impide guardar o actualizar registros con ID vacío, nombre vacío o edad inválida.	UC5
-------	------------------------------	---------	------	--	-----

- Diagramas de Casos de Uso

```

@startuml
left to right direction
skinparam packageStyle rectangle
skinparam shadowing false
skinparam usecase {
    BackgroundColor #E8F0FE
    BorderColor #2F5597
}
skinparam actor {
    BorderColor black
    BackgroundColor #FDFDFD
}

actor "Administrador Académico" as Admin

rectangle "Sistema CRUD de Estudiantes" {

    usecase "Agregar Estudiante" as UC1
    usecase "Actualizar Estudiante" as UC2
    usecase "Eliminar Estudiante" as UC3
    usecase "Mostrar Estudiantes" as UC4
    usecase "Validar Datos" as UC5

}

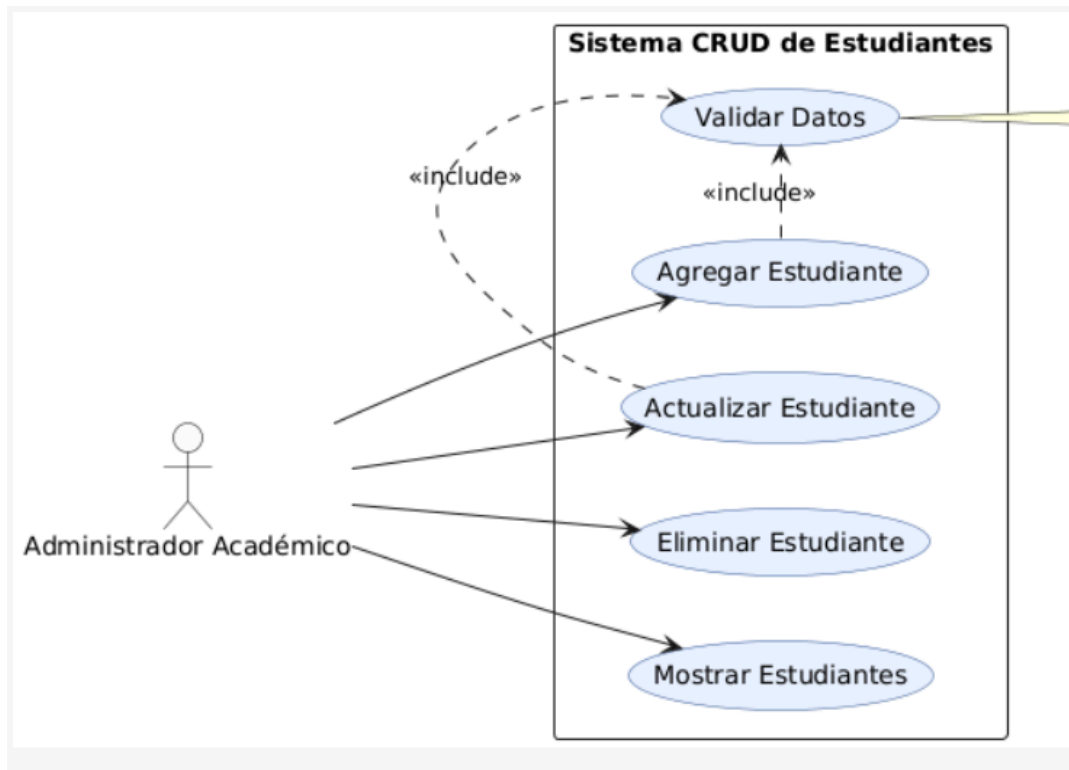
Admin --> UC1
Admin --> UC2
Admin --> UC3
Admin --> UC4

UC1 .> UC5 : <<include>>
UC2 .> UC5 : <<include>>

note right of UC5
Datos mínimos según interfaz:
- ID
- Nombre
- Edad
end note

```

@enduml



@startuml

left to right direction

skinparam packageStyle rectangle

skinparam shadowing false

actor "Administrador académico" as Admin

actor "Sistema" as Sistema

```
rectangle "Sistema de Gestión de Estudiantes" {  
    usecase "Agregar estudiante" as UC1  
    usecase "Actualizar estudiante" as UC2  
    usecase "Eliminar estudiante" as UC3  
    usecase "Mostrar todos\nlos estudiantes" as UC4  
    usecase "Validar datos\ndel estudiante" as UC5  
}
```

Admin --> UC1

Admin --> UC2

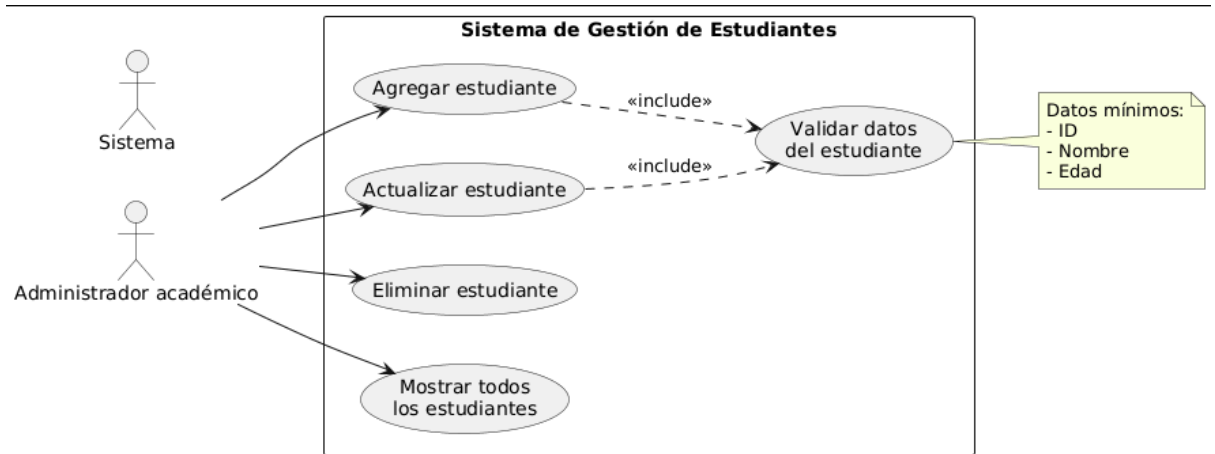
Admin --> UC3

Admin --> UC4

UC1 ..> UC5 : <<include>>

UC2 ..> UC5 : <<include>>

note right of UC5
 Datos mínimos:
 - ID
 - Nombre
 - Edad
 end note
 @enduml

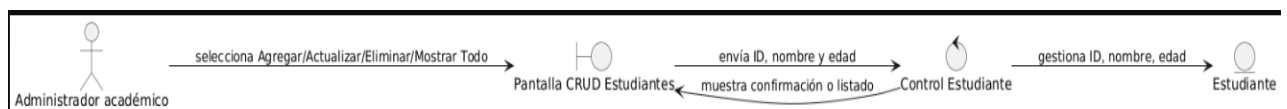


- Puente

@startuml
 left to right direction

actor "Administrador académico" as Admin
 boundary "Pantalla CRUD Estudiantes" as UI
 control "Control Estudiante" as Ctrl
 entity "Estudiante" as Est

Admin --> UI : selecciona Agregar/Actualizar/Eliminar/Mostrar Todo
 UI --> Ctrl : envía ID, nombre y edad
 Ctrl --> Est : gestiona ID, nombre, edad
 Ctrl --> UI : muestra confirmación o listado
 @enduml



- Conclusión Técnica

La elaboración del diagrama de casos de uso permitió comprender de forma práctica cómo transformar requisitos funcionales en modelos visuales claros y organizados. Durante el desarrollo del taller se fortaleció la capacidad de identificar actores, funcionalidades principales y relaciones entre procesos dentro del sistema CRUD de estudiantes. Además, el uso de PlantUML facilitó la creación técnica y ordenada del diagrama, demostrando que esta herramienta mejora la documentación y comunicación del análisis de software. En conclusión, la actividad fue útil para afianzar conceptos fundamentales de modelado y trazabilidad en la etapa inicial del desarrollo de sistemas.

- Recomendaciones

Realizar un levantamiento claro y completo de los requisitos funcionales antes de iniciar el modelado, para asegurar que el diagrama represente las necesidades reales del sistema.

Mantener consistencia en la nomenclatura de actores y casos de uso, utilizando nombres orientados a acciones y objetivos del usuario.

Validar periódicamente la trazabilidad entre requisitos, actores y casos de uso para evitar omisiones o funcionalidades duplicadas.

GUÍA 02

- Objetivo

Analizar de forma progresiva un CRUD de Estudiante mediante tres modelos conectados —casos de uso, clases de análisis y diagramas de secuencia— con el propósito de comprender qué responsabilidad cumple cada elemento del modelo y cómo se conserva la trazabilidad desde el requisito funcional hasta la interacción entre objetos, sin enfocarse en la programación de la interfaz.

- Desarrollo
 - Diagrama de Clases

```
@startuml
skinparam classAttributeIconSize 0
skinparam shadowing false
left to right direction
```

```
class "FormularioCrudEstudiante" <<boundary>> {
    - txtId: int
    - txtNombre: String
    - txtEdad: int
    + clickAgregar()
    + clickActualizar()
    + clickEliminar()
    + clickMostrarTodo()
    + mostrarMensaje(mensaje: String)
    + mostrarTabla(estudiantes: List<Estudiante>)
}

class "ControlEstudiante" <<control>> {
    + agregarEstudiante(id: int, nombre: String, edad: int): Resultado
    + actualizarEstudiante(id: int, nombre: String, edad: int): Resultado
    + eliminarEstudiante(id: int): Resultado
    + mostrarTodos(): List<Estudiante>
    + validarDatos(id: int, nombre: String, edad: int): boolean
}

class "Estudiante" <<entity>> {
    - id: int
    - nombre: String
    - edad: int
    + Estudiante(id: int, nombre: String, edad: int)
    + getId(): int
    + getNombre(): String
    + getEdad(): int
}

class "RepositorioEstudiante" <<repository>> {
    + existId(id: int): boolean
    + guardar(estudiante: Estudiante): void
    + buscarPorId(id: int): Estudiante
    + actualizar(estudiante: Estudiante): void
    + eliminar(id: int): void
    + listarTodos(): List<Estudiante>
```

```
}

```

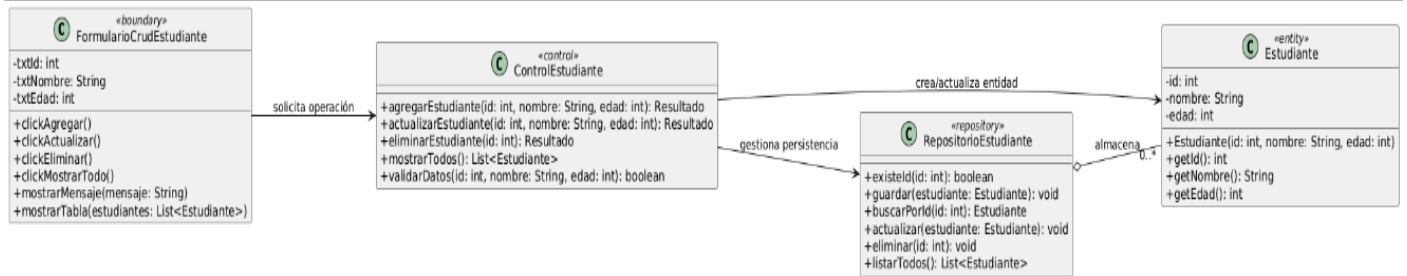
FormularioCrudEstudiante --> ControlEstudiante : solicita operación

ControlEstudiante --> Estudiante : crea/actualiza entidad

ControlEstudiante --> RepositorioEstudiante : gestiona persistencia

RepositorioEstudiante o-- "0..*" Estudiante : almacena

@enduml

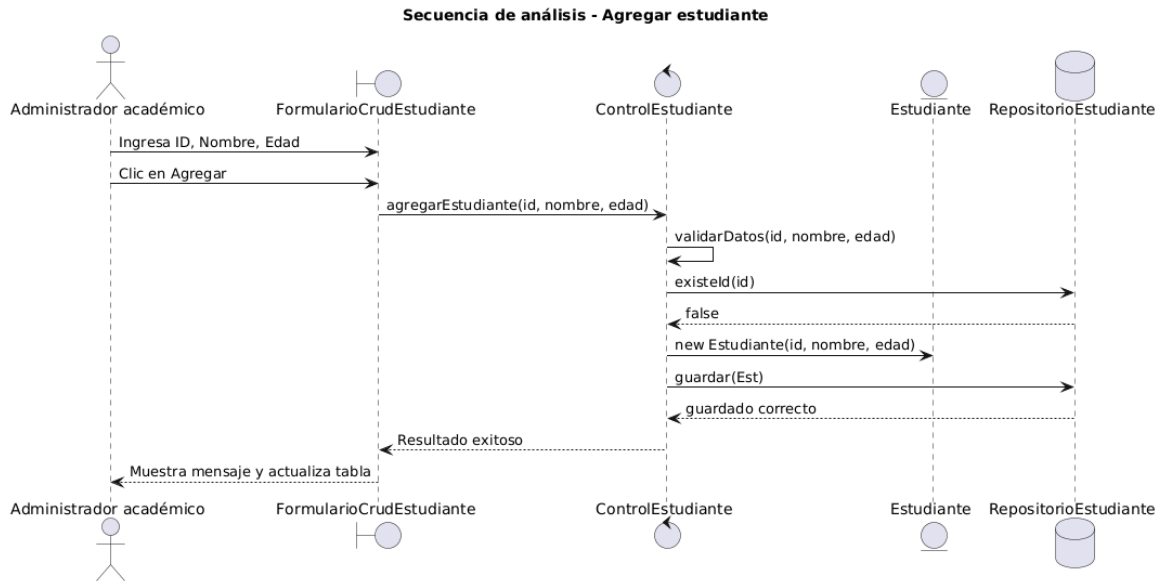


- Diagrama de Secuencia
 - Agregar Estudiante

```

@startuml
title Secuencia de análisis - Agregar estudiante
actor "Administrador académico" as Admin
boundary "FormularioCrudEstudiante" as UI
control "ControlEstudiante" as Ctrl
entity "Estudiante" as Est
database "RepositorioEstudiante" as Repo

Admin -> UI : Ingresa ID, Nombre, Edad
Admin -> UI : Clic en Agregar
UI -> Ctrl : agregarEstudiante(id, nombre, edad)
Ctrl -> Ctrl : validarDatos(id, nombre, edad)
Ctrl -> Repo : existeId(id)
Repo --> Ctrl : false
Ctrl -> Est : new Estudiante(id, nombre, edad)
Ctrl -> Repo : guardar(Est)
Repo --> Ctrl : guardado correcto
Ctrl --> UI : Resultado exitoso
UI --> Admin : Muestra mensaje y actualiza tabla
@enduml
  
```

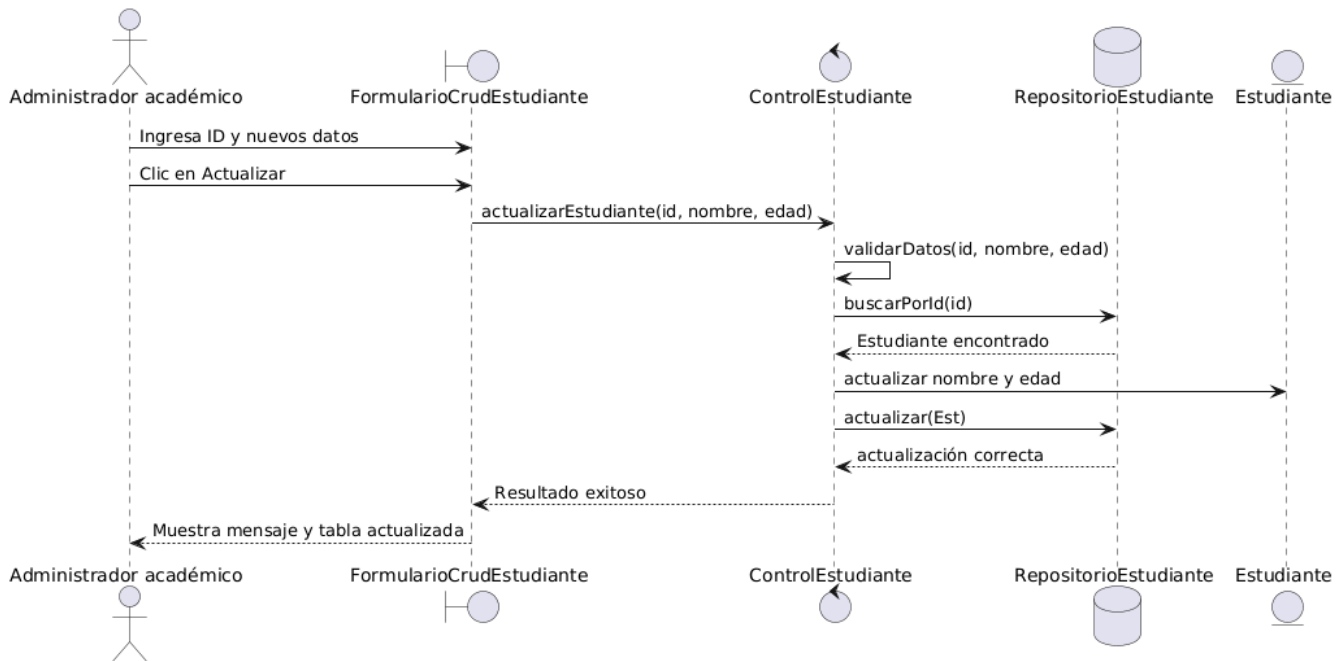
■ Actualizar Estudiante

```

@startuml
title Secuencia de análisis - Actualizar estudiante
actor "Administrador académico" as Admin
boundary "FormularioCrudEstudiante" as UI
control "ControlEstudiante" as Ctrl
database "RepositorioEstudiante" as Repo
entity "Estudiante" as Est

Admin -> UI : Ingresa ID y nuevos datos
Admin -> UI : Clic en Actualizar
UI -> Ctrl : actualizarEstudiante(id, nombre, edad)
Ctrl -> Ctrl : validarDatos(id, nombre, edad)
Ctrl -> Repo : buscarPorId(id)
Repo --> Ctrl : Estudiante encontrado
Ctrl -> Est : actualizar nombre y edad
Ctrl -> Repo : actualizar(Est)
Repo --> Ctrl : actualización correcta
Ctrl --> UI : Resultado exitoso
UI --> Admin : Muestra mensaje y tabla actualizada
@enduml
  
```

Secuencia de análisis - Actualizar estudiante

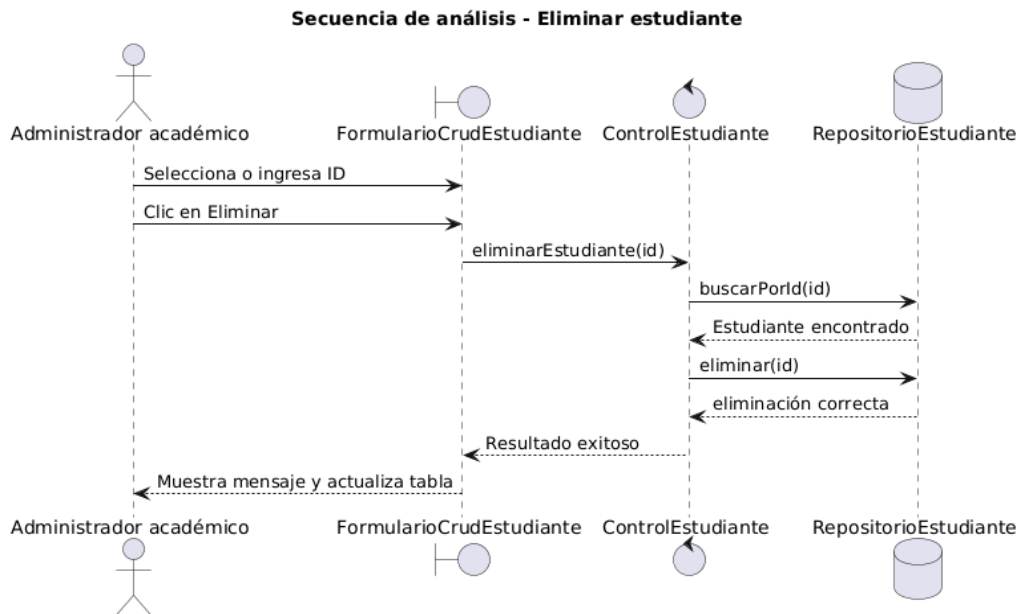


■ Eliminar Estudiante

```

@startuml
title Secuencia de análisis - Eliminar estudiante
actor "Administrador académico" as Admin
boundary "FormularioCrudEstudiante" as UI
control "ControlEstudiante" as Ctrl
database "RepositorioEstudiante" as Repo

Admin -> UI : Selecciona o ingresa ID
Admin -> UI : Clic en Eliminar
UI -> Ctrl : eliminarEstudiante(id)
Ctrl -> Repo : buscarPorId(id)
Repo --> Ctrl : Estudiante encontrado
Ctrl -> Repo : eliminar(id)
Repo --> Ctrl : eliminación correcta
Ctrl --> UI : Resultado exitoso
UI --> Admin : Muestra mensaje y actualiza tabla
@enduml
    
```



- **Conclusión Técnica**

Al revisar el material propuesto para el taller , se pudo apreciar un ejemplo, de cómo sería la forma práctica, concretamente, cómo la trazabilidad entre modelos garantiza la coherencia del análisis. Partir de los requisitos funcionales y casos de uso permitió definir sin ambigüedad qué necesita el actor, mientras que el uso de la relación <<include>> para "Validar datos" evitó duplicar lógica en las operaciones de Agregar y Actualizar.

La derivación de clases de análisis con estereotipos boundary, control, entity y repository obligó a separar responsabilidades desde el inicio, impidiendo mezclar lógica de negocio con la interfaz. La principal dificultad fue mantener la disciplina de que cada mensaje en los diagramas de secuencia tuviera su operación correspondiente declarada en el diagrama de clases, lo que exigió revisar y ajustar varias veces según la lista de verificación de coherencia.

En conclusión, se interiorizó que el análisis no se construye con piezas aisladas, sino como una cadena donde cada decisión en un modelo impacta al siguiente. Esta metodología, aunque rigurosa, reduce la ambigüedad y prepara de forma sólida el tránsito hacia la etapa de diseño.

- **Recomendaciones**

- En el diagrama de clases, cambiar todas las referencias del id y la edad por int en lugar de strings.
- No mezclar detalles de implementación en los diagramas de análisis; mantener los estereotipos boundary, control, entity y repository con responsabilidades claras para no anticipar decisiones técnicas prematuras.
- Revisar la sintaxis de PlantUML antes de generar las imágenes finales, ya que errores como el guion simple en lugar de la flecha (-->) pueden romper la semántica de los diagramas de secuencia y afectar la presentación.

